Boox Store administrator: code and setup guide

This guide explains the private admin area, where its code is, how the first admin is created, and how to manage books and other admins. It intentionally does not contain any real password.

Admin page

- Deployed page after this version is deployed: <https://bookstore-practicum.onrender.com/backroom/books>
- Local Vite page: <http://localhost:5173/backroom/books>
- First administrator username: Louis
- Password: set your own strong password privately; it must be at least 12 characters.

The page is not linked from the public bookstore navigation. The URL is not the security boundary: the server checks the signed-in user's role for every admin API request. Knowing the link does not give a customer admin access.

Files changed or added

File	Purpose
frontend/src/components/AdminPage.jsx	Admin login, book search, add/edit/price controls, remove/restore controls, and create-admin form.
frontend/src/App.jsx	Shows AdminPage only at /backroom/books; other paths continue to show the storefront.
frontend/src/lib/auth-client.js	Enables the Better Auth username sign-in method in React.
server/src/admin-routes.js	Server endpoints for listing and changing books, and creating another admin. Every route requires an admin session.
server/src/admin-validation.js	Checks book fields and prices before database changes.
server/src/admin-validation.test.js	Tests for valid book data, price precision, category, and cover URL validation.

File	Purpose
server/src/admin-bootstrap.js	One-time command that creates the first admin, or promotes the configured existing account.
server/src/auth.js	Adds username sign-in and requires new passwords to be at least 12 characters.
server/src/db/auth-schema.js	Adds username, displayUsername, and role to the existing Better Auth user table.
server/src/db/schema.js	Adds a book availability flag used to hide removed books without deleting order history.
server/src/db/migrations/0003_add_admin_accounts.sql	Adds username and role columns, a unique username index, and book availability.
server/src/index.js	Mounts the protected admin API, hides unavailable books from public browsing, and serves the admin page on refresh.
server/src/checkout-routes.js	Prevents checkout from buying a book that has been removed from the store.
frontend/src/components/AccountSection.jsx	Aligns new customer signups with the stronger 12-character minimum. Existing users can still sign in with their existing password.
server/.env.example	Shows the names of the optional, private one-time admin bootstrap settings.
server/package.json	Adds npm run admin:bootstrap.

How the security works

1. Better Auth verifies the username and password and creates a normal authenticated session.
2. Each request to /api/admin/... calls Better Auth to read that session.
3. The server allows the action only when the database user has role = admin.
4. Public sign-up cannot set the role; it defaults to customer.
5. The private page itself is only a convenient sign-in screen. Even a direct API request still needs an administrator session.

The first account is provisioned with private environment variables. Once it exists, signed-in admins can use Create admin account to add more admins.

The new account gets its own username, email, and password and is promoted to admin only by the protected server route.

Create the first admin

The migration has been applied to the configured database. To set up another database, run migration server/src/db/migrations/0003_add_admin_accounts.sql once against that database first.

In the server/.env file, set these values privately:

```
ADMIN_USERNAME=Louis
ADMIN_EMAIL=your email address
ADMIN_PASSWORD=your own strong password with at least 12 characters
```

Use the same DATABASE_URL as the website. Do not paste the password into chat, README, source code, or this guide. Then run these commands in PowerShell:

```
cd C:\Users\louis\Documents\bookstore-practicum\server
npm.cmd run admin:bootstrap
```

When it reports that the administrator account is ready, remove ADMIN_PASSWORD from server/.env. The bootstrap command never prints the password. If the configured email already has a Better Auth account, the command can promote it, but it does not change that account's existing password.

If you change your password in the future, use the normal account password management process; do not keep the bootstrap password in environment settings.

What book changes do

- Add a book: validates the title, author, category, description, price, and image path before inserting the new catalogue record.
- Edit a book: saves those same fields. New checkout sessions use the updated database price. Previous order items keep their original price snapshot.
- Remove from store: marks the book unavailable. It disappears from public book listings and new checkouts but remains in the database so past order records are not broken.
- Restore to store: marks a removed book available again.

Cover images can use an existing /images/... path or a secure HTTPS URL. Uploaded image files are not part of this admin feature.

Core code excerpts

The Better Auth configuration adds the username plugin and a protected customer-default role:

```
emailAndPassword: {
  enabled: true,
  minPasswordLength: 12,
},
user: {
  additionalFields: {
    role: {
      type: 'string',
      defaultValue: 'customer',
      input: false,
    },
  },
},
plugins: [username()],
```

The admin API checks the server-side session role before handling admin requests:

```
const session = await auth.api.getSession({
  headers: fromNodeHeaders(req.headers),
})

if (session?.user?.role !== 'admin') {
  return res.status(session?.user ? 403 : 401).json({
    error: 'Administrator access is required.',
  })
}
```

The public catalogue and checkout both filter out removed books:

```
.where(eq(books.isAvailable, true))
```

The page checks the private path in frontend/src/App.jsx:

```
if (window.location.pathname === '/backroom/books') {
  return <AdminPage />
}
```

This path selection only decides which page is displayed. The role checks in server/src/admin-routes.js are what protect the data and actions.

Validation and current state

The admin password must be at least 12 characters. Book prices must be positive and use no more than two decimal places. Category and cover image values are validated on the server, not just in the browser.

The first-admin account is not created by this code change until the private ADMIN_EMAIL and strong ADMIN_PASSWORD are set and

npm.cmd run admin:bootstrap is run. The code never embeds a password in the PDF or repository.